

Forward Kinematics of an $R_{\perp}R_{\parallel}R$ Robot: Homogeneous Transformations vs Dual Quaternions

Jatin Satyam
Robotic System - I
 Email: jsatyam@asu.edu

Abstract—This paper presents a comprehensive forward-kinematics analysis of an $R_{\perp}R_{\parallel}R$ robotic manipulator using two parallel formulations: homogeneous transformation matrices and dual-quaternion-based kinematics. The objective is to compute the end-effector pose using both approaches and verify that the numerical outputs—position and orientation—are equivalent to machine precision. The results demonstrate that both methods yield identical solutions within numerical tolerance ($\sim 10^{-14}$ mm for position), confirming the correctness of the derived analytical expressions and the reliability of the DQ Robotics toolbox implementation.

Index Terms—Forward kinematics, homogeneous transformations, dual quaternions, robotic manipulator, $R_{\perp}R_{\parallel}R$ robot

I. INTRODUCTION

Forward kinematics is a fundamental problem in robotics that determines the position and orientation of a robot's end-effector given its joint parameters. This analysis is essential for robot control, path planning, and simulation. Multiple mathematical frameworks exist for representing and computing forward kinematics, each with distinct advantages in terms of computational efficiency, singularity handling, and geometric insight.

This paper examines an $R_{\perp}R_{\parallel}R$ serial manipulator—a three-degree-of-freedom robot consisting of three revolute joints where the first joint's rotation axis is perpendicular to the second, and the second and third joints have parallel rotation axes. Two computational approaches are employed: classical homogeneous transformation matrices and the modern dual quaternion formalism.

The contributions of this work include:

- Derivation of closed-form forward kinematics using homogeneous transformations
- Implementation using dual quaternion algebra via the DQ Robotics toolbox
- Numerical validation demonstrating equivalence between both methods
- Complete MATLAB implementation for reproducibility

II. ROBOT GEOMETRY AND PARAMETERS

A. Joint Configuration

The robot consists of a sequence of three revolute joints with the following structure:

- **Joint 1:** Revolute joint with a twist of -90
- **Joint 2:** Revolute joint, aligned parallel to Joint 3
- **Joint 3:** Revolute joint

B. Link Parameters

The relevant Denavit-Hartenberg (DH) parameters and link dimensions are:

$$a_1 = 53.5 \text{ mm} \quad (1)$$

$$a_2 = 150 \text{ mm} \quad (2)$$

$$a_3 = 150 \text{ mm} \quad (3)$$

$$L_x = 90 \text{ mm} \text{ (offset)} \quad (4)$$

These parameters define the geometric relationship between consecutive joint frames and are used consistently across both computational approaches.

III. HOMOGENEOUS TRANSFORMATION METHOD

A. Link Transformations

Homogeneous transformation matrices provide a compact representation of rotation and translation between coordinate frames. For the $R_{\perp}R_{\parallel}R$ robot, three link transforms are defined:

Transform from base to Joint 1:

$$H_{01} = \begin{bmatrix} \cos \theta_1 & 0 & -\sin \theta_1 & 0 \\ \sin \theta_1 & 0 & \cos \theta_1 & 0 \\ 0 & -1 & 0 & a_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Transform from Joint 1 to Joint 2:

$$H_{12} = \begin{bmatrix} -\sin \theta_2 & -\cos \theta_2 & 0 & L_x + a_2 \sin \theta_2 \\ \cos \theta_2 & -\sin \theta_2 & 0 & -a_2 \cos \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Transform from Joint 2 to Joint 3:

$$H_{23} = \begin{bmatrix} \cos(\theta_3 - \theta_2) & -\sin(\theta_3 - \theta_2) & 0 & a_3 \sin(\theta_3 - \theta_2) \\ \sin(\theta_3 - \theta_2) & \cos(\theta_3 - \theta_2) & 0 & -a_3 \cos(\theta_3 - \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

B. Complete Forward Kinematics

The overall transformation is obtained through matrix multiplication:

$$H_{03} = H_{01} \cdot H_{12} \cdot H_{23} \quad (8)$$

This simplifies to the following closed forms for rotation and position:

Rotation matrix:

$$R_{03} = \begin{bmatrix} -\sin \theta_3 \cos \theta_1 & -\cos \theta_1 \cos \theta_3 & -\sin \theta_1 \\ -\sin \theta_1 \sin \theta_3 & -\sin \theta_1 \cos \theta_3 & \cos \theta_1 \\ -\cos \theta_3 & \sin \theta_3 & 0 \end{bmatrix} \quad (9)$$

Position vector (base frame):

$$\mathbf{p}_{03}^{\text{base}} = \begin{bmatrix} (a_2 \sin \theta_2 + a_3 \cos \theta_3 + L_x) \cos \theta_1 \\ (a_2 \sin \theta_2 + a_3 \cos \theta_3 + L_x) \sin \theta_1 \\ a_1 + a_2 \cos \theta_2 - a_3 \sin \theta_3 \end{bmatrix} \quad (10)$$

C. Laboratory Frame Correction

The laboratory setup measures Z from the top of link 1. Therefore, an adjusted position vector is used:

$$\mathbf{p}_{03}^{\text{lab}} = \begin{bmatrix} p_{03,x}^{\text{base}} \\ p_{03,y}^{\text{base}} \\ a_2 \cos \theta_2 - a_3 \sin \theta_3 \end{bmatrix} \quad (11)$$

where the Z-component is $p_{03,z}^{\text{base}} - a_1$.

IV. DUAL QUATERNION METHOD

A. Dual Quaternion Representation

A dual quaternion represents both rotation and translation in a unified algebraic structure:

$$\mathbf{x} = r + \frac{\varepsilon}{2} tr \quad (12)$$

where r is a unit quaternion representing rotation, t is a pure quaternion representing translation, and ε is the dual unit ($\varepsilon^2 = 0$).

B. Link Dual Quaternions

Rotation Quaternions:

$$r_{01} = R_z(\theta_1) \cdot R_x(-90) \quad (13)$$

$$r_{12} = R_z(\theta_2 + 90) \quad (14)$$

$$r_{23} = R_z(\theta_3 - \theta_2) \quad (15)$$

Translation Vectors:

$$\mathbf{t}_{01} = [0, 0, a_1]^T \quad (16)$$

$$\mathbf{t}_{12} = [L_x + a_2 \sin \theta_2, -a_2 \cos \theta_2, 0]^T \quad (17)$$

$$\mathbf{t}_{23} = [a_3 \sin(\theta_3 - \theta_2), -a_3 \cos(\theta_3 - \theta_2), 0]^T \quad (18)$$

Link Dual Quaternions:

$$\mathbf{x}_{01} = r_{01} + \frac{\varepsilon}{2} \mathbf{t}_{01} r_{01} \quad (19)$$

$$\mathbf{x}_{12} = r_{12} + \frac{\varepsilon}{2} \mathbf{t}_{12} r_{12} \quad (20)$$

$$\mathbf{x}_{23} = r_{23} + \frac{\varepsilon}{2} \mathbf{t}_{23} r_{23} \quad (21)$$

C. Complete Pose Computation

The complete end-effector pose is obtained through dual quaternion multiplication:

$$\mathbf{x}_{03} = \mathbf{x}_{01} \cdot \mathbf{x}_{12} \cdot \mathbf{x}_{23} \quad (22)$$

Translation and rotation components are extracted using the DQ Robotics toolbox functions `translation()` and `rotation()`, then converted to Cartesian coordinates and rotation matrices for comparison with the homogeneous transformation results.

V. NUMERICAL RESULTS

A. Test Configuration

The forward kinematics was evaluated at the zero configuration:

$$\boldsymbol{\theta} = [0, 0, 0]^T \quad (23)$$

B. Expected Results

At zero joint angles, the expected laboratory-frame end-effector position is:

$$\mathbf{p}_{\text{expected}} = [240, 0, 150]^T \text{ mm} \quad (24)$$

This can be verified geometrically: with all joints at zero, the end-effector extends along the X-axis by $L_x + a_2 + a_3 = 90 + 150 + 0 = 240$ mm, remains at $Y = 0$, and is at height $a_2 = 150$ mm above the top of link 1.

C. Computational Results

Both methods produced identical results within numerical precision:

Homogeneous Transformation Result:

$$\mathbf{p}_{\text{HT}}^{\text{lab}} = [240.0000, 0.0000, 150.0000]^T \text{ mm} \quad (25)$$

Dual Quaternion Result:

$$\mathbf{p}_{\text{DQ}}^{\text{lab}} = [240.0000, 0.0000, 150.0000]^T \text{ mm} \quad (26)$$

D. Error Analysis

The differences between the two methods are:

$$\|\mathbf{p}_{\text{HT}} - \mathbf{p}_{\text{DQ}}\| \approx 10^{-14} \text{ mm} \quad (27)$$

$$\Delta \theta_{\text{orientation}} = 0 \text{ rad} \quad (28)$$

These errors are attributable solely to floating-point arithmetic round-off and confirm numerical equivalence.

VI. DISCUSSION

A. Method Comparison

Both homogeneous transformations and dual quaternions offer distinct advantages:

Homogeneous Transformations:

- Widely adopted and well-understood in robotics
- Direct geometric interpretation
- Standard representation in most robotics textbooks
- 4x4 matrix representation is computationally straightforward

Dual Quaternions:

- More compact representation (8 parameters vs. 16)
- Natural handling of screw motions
- Efficient for interpolation and velocity computation
- Avoids gimbal lock issues in orientation representation

B. Validation Significance

The numerical agreement between methods serves multiple purposes:

- 1) Validates the correctness of the derived analytical expressions
- 2) Confirms proper implementation of both computational frameworks
- 3) Verifies the DQ Robotics toolbox produces consistent results
- 4) Provides confidence in using either method for control applications

C. Computational Efficiency

For forward kinematics, both methods exhibit similar computational complexity. However, dual quaternions may offer advantages in:

- Inverse kinematics formulations
- Trajectory planning with orientation interpolation
- Velocity and Jacobian computations

VII. CONCLUSION

This paper presented a complete forward-kinematics analysis of an $R_{\perp}R_{\parallel}R$ robotic manipulator using two mathematical frameworks: homogeneous transformation matrices and dual quaternions. Both approaches were implemented in MATLAB and produced identical results within machine precision ($\sim 10^{-14}$ mm position error, 0 rad orientation error).

The numerical validation confirms:

- Correctness of the derived closed-form kinematics equations
- Reliability of the DQ Robotics toolbox implementation
- Equivalence of both computational methods for this robot geometry

Future work could extend this analysis to:

- Inverse kinematics solutions using both frameworks
- Differential kinematics and Jacobian computation
- Dynamic modeling incorporating mass properties
- Experimental validation on physical hardware

The complete MATLAB implementation is provided in Appendix A for reproducibility and further investigation.

ACKNOWLEDGMENT

The author would like to thank the robotics laboratory staff for providing access to the $R_{\perp}R_{\parallel}R$ manipulator and computational resources.

REFERENCES

- [1] J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," *Journal of Applied Mechanics*, vol. 22, no. 2, pp. 215–221, 1955.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2017.
- [3] B. V. Adorno, "Two-arm manipulation: From manipulators to enhanced human-robot collaboration," Ph.D. dissertation, Université Montpellier 2, 2011.
- [4] K. Dantam, "Robust and efficient forward, differential, and inverse kinematics using dual quaternions," *International Journal of Robotics Research*, vol. 40, no. 10–11, pp. 1087–1105, 2021.

- [5] F. S. Grassia, "Practical parameterization of rotations using the exponential map," *Journal of Graphics Tools*, vol. 3, no. 3, pp. 29–48, 1998.
- [6] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge University Press, 2017.
- [7] B. V. Adorno and J. Y. Ishihara, "The DQ Robotics Library," *IEEE Robotics & Automation Magazine*, 2023. [Online]. Available: <https://github.com/dqrobotics>
- [8] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.

APPENDIX

The complete MATLAB code for computing forward kinematics using both methods is provided below.

A. Main Script

```
1 clear; clc; close all;
2
3 % Robot parameters
4 a1 = 53.5; % mm
5 a2 = 150; % mm
6 a3 = 150; % mm
7 Lx = 90; % mm
8
9 % Joint angles (degrees)
10 theta1_deg = 0;
11 theta2_deg = 0;
12 theta3_deg = 0;
13
14 % Convert to radians
15 theta1 = deg2rad(theta1_deg);
16 theta2 = deg2rad(theta2_deg);
17 theta3 = deg2rad(theta3_deg);
18
19 % Precompute trigonometric values
20 c1 = cos(theta1); s1 = sin(theta1);
21 c2 = cos(theta2); s2 = sin(theta2);
22 c3 = cos(theta3); s3 = sin(theta3);
23 c32 = cos(theta3-theta2);
24 s32 = sin(theta3-theta2);
```

Listing 1. Forward Kinematics Computation

B. Homogeneous Transformation Method

```
1 % Homogeneous transformations
2 H01 = [ c1  0 -s1  0;
3         s1  0  c1  0;
4         0 -1  0 a1;
5         0  0  0  1];
6
7 H12 = [-s2 -c2  0 Lx + a2*s2;
8         c2 -s2  0 -a2*c2;
9         0  0  1  0;
10        0  0  0  1];
11
12 H23 = [c32 -s32  0 a3*s32;
13        s32  c32  0 -a3*c32;
14        0  0  1  0;
15        0  0  0  1];
16
17 % Complete transformation
18 H03 = H01*H12*H23;
19
20 % Extract rotation and position
21 R_HT = H03(1:3,1:3);
22 p_HT = H03(1:3,4);
23
24 % Lab frame correction
25 H03_lab = H03;
26 H03_lab(3,4) = H03_lab(3,4) - a1;
27 p_HT_lab = p_HT;
28 p_HT_lab(3) = p_HT_lab(3) - a1;
```

Listing 2. HT Forward Kinematics

C. Dual Quaternion Method

```

1 % Initialize DQ Robotics
2 include_namespace_dq;
3
4 % Rotation helper functions
5 rz = @(ang) (cos(ang/2) + k_*sin(ang/2));
6 rx = @(ang) (cos(ang/2) + i_*sin(ang/2));
7
8 % Rotation quaternions
9 r01 = rz(theta1) * rx(-pi/2);
10 r12 = rz(theta2 + pi/2);
11 r23 = rz(theta3 - theta2);
12
13 % Translation vectors
14 t01 = a1 * k_;
15 t12 = (Lx + a2*s2)*i_ + (-a2*c2)*j_;
16 t23 = (a3*s32)*i_ + (-a3*c32)*j_;
17
18 % Dual quaternions
19 x01 = r01 + 0.5*E_*t01*r01;
20 x12 = r12 + 0.5*E_*t12*r12;
21 x23 = r23 + 0.5*E_*t23*r23;
22
23 % Complete pose
24 x03 = x01 * x12 * x23;
25
26 % Extract translation and rotation
27 t_DQ = translation(x03);
28 r_DQ = rotation(x03);
29
30 % Convert quaternion to rotation matrix
31 q = r_DQ.q;
32 w = q(1); x = q(2); y = q(3); z = q(4);
33
34 R_DQ = [ 1-2*(y^2+z^2), 2*(x*y - w*z), 2*(x*z + w*y);
35          2*(x*y + w*z), 1-2*(x^2+z^2), 2*(y*z - w*x);
36          2*(x*z - w*y), 2*(y*z + w*x), 1-2*(x^2+y^2) ];
37
38 % Extract position
39 p_DQ = [ t_DQ.q(2); t_DQ.q(3); t_DQ.q(4) ];
40
41 % Lab frame correction
42 p_DQ_lab = p_DQ;
43 p_DQ_lab(3) = p_DQ_lab(3) - a1;

```

Listing 3. DQ Forward Kinematics

D. Error Analysis

```

1 % Compute errors
2 pos_err_lab = p_HT_lab - p_DQ_lab;
3 pos_err_norm = norm(pos_err_lab);
4
5 R_err = R_HT * R_DQ.';
6 angle_err = acos(max(-1, min(1, ...
7     (trace(R_err)-1)/2)));
8
9 % Display results
10 fprintf('\n==== Forward Kinematics =====\n');
11 fprintf('(Lab frame: Z from top of link-1)\n');
12 fprintf('Input angles (deg): theta1=%.2f, ', ...
13     theta2_deg, theta3_deg);
14 fprintf('theta2=%.2f, theta3=%.2f\n', ...
15     theta1_deg, theta2_deg, theta3_deg);
16
17 disp('H03_lab =');
18 disp(H03_lab);
19
20 fprintf('p_HT_lab (mm) = [%4f %4f %4f]^T\n', ...
21     p_HT_lab(1), p_HT_lab(2), p_HT_lab(3));
22
23 disp('R_HT =');
24 disp(R_HT);
25
26 fprintf('\n--- Dual Quaternion Result ---\n');
27 fprintf('p_DQ_lab (mm) = [%4f %4f %4f]^T\n', ...
28     p_DQ_lab(1), p_DQ_lab(2), p_DQ_lab(3));
29
30 disp('R_DQ =');
31 disp(R_DQ);
32

```

```

33 fprintf('\n--- Differences (HT - DQ) ---\n');
34 fprintf('Position error (mm): '...
35     ' [%3e %3e %3e]\n', ...
36     pos_err_lab(1), pos_err_lab(2), ...
37     pos_err_lab(3));
38 fprintf('||position error|| (mm): %3e\n', ...
39     pos_err_norm);
40 fprintf('Orientation error (rad): %3e\n', ...
41     angle_err);
42 fprintf('Orientation error (deg): %3e\n', ...
43     rad2deg(angle_err));

```

Listing 4. Error Computation and Display